

Claude Code Training

Press Space for next page →

Contact Info

Ken Kousen

Kousen IT, Inc.

- ken.kousen@kousenit.com
- <http://www.kousenit.com>
- <http://kousenit.org> (blog)
- Social Media:
 - [@kenkousen](#) (twitter)
 - [@kenkousen@foojay.social](#) (mastodon)
 - [@kousenit.com](#) (bluesky)
- *Tales from the jar side* (free newsletter)
 - <https://kenkousen.substack.com>
 - <https://youtube.com/@talesfromthejarside>

Course Overview

- **Duration:** 5 hours of hands-on learning
- **Format:** Instructor-led with multiple labs
- **Hands-on Labs:** Real codebases in Python, JavaScript, Java
- **Prerequisites:** Command-line experience, development background

Topics Covered

- **Foundation:** Installation, surfaces, CLI basics, code exploration
- **Core Skills:** Testing, documentation, git operations
- **Customization:** CLAUDE.md, skills, hooks, output styles
- **Extensibility:** Plugins, MCP integration
- **Advanced:** Effort Levels, Plan Mode, Ultraplan, Subagents, Agent Teams, SDKs

Pricing Plans

- **Pro Plan** - \$20/month
 - ~10-40 prompts per 5 hours
 - Sonnet 4.6 only
- **Max Plan 5x** - \$100/month
 - ~50-200 prompts per 5 hours
 - Sonnet 4.6 or Opus 4.6
- **Max Plan 20x** - \$200/month
 - ~200-800 prompts per 5 hours
 - Sonnet 4.6 or Opus 4.6
- **Note:** Opus 4.6 uses 5x more credits than Sonnet 4.6
- **Limits reset:** Every 5 hours

📖 **Full details:** Using Claude Code with your Pro or Max plan

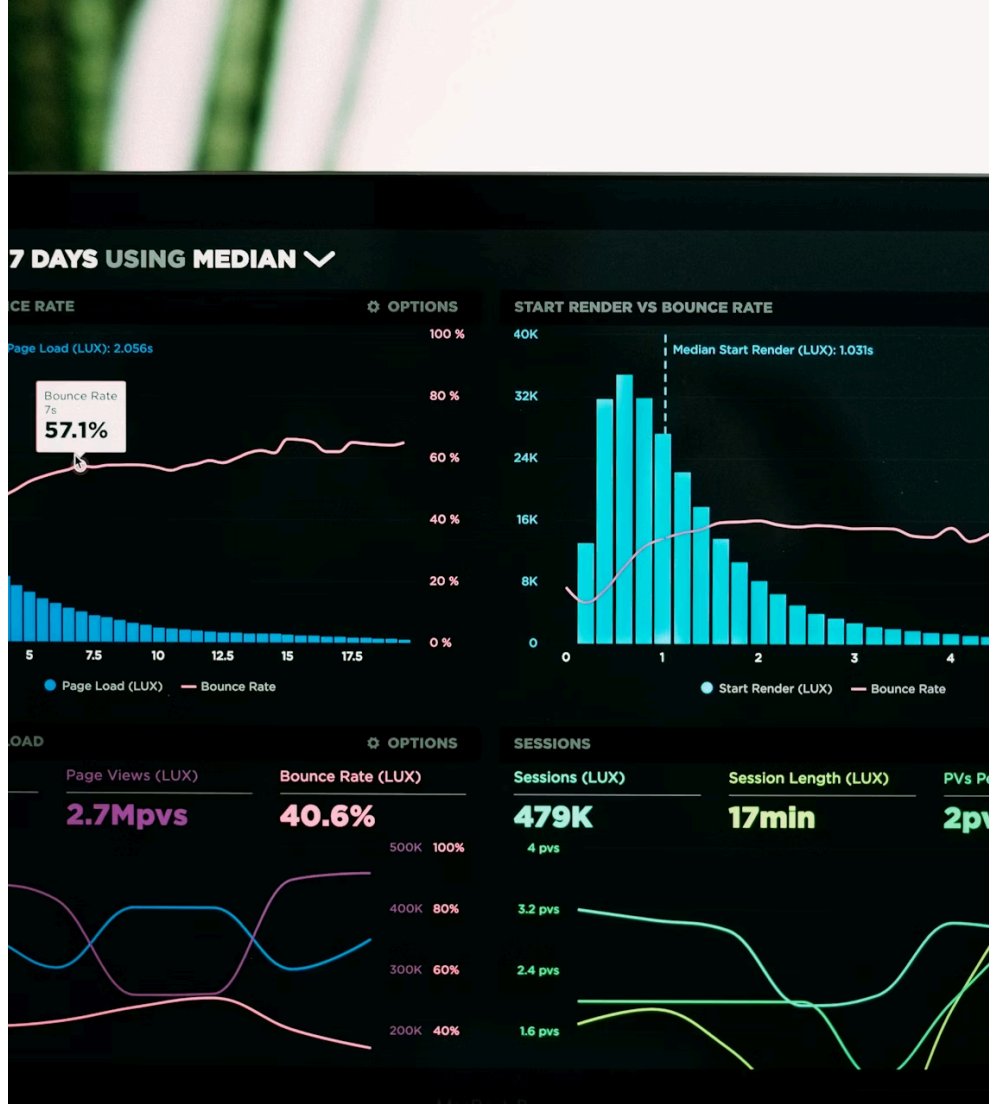
What is Claude Code?

- AI development tool across **5 surfaces**: CLI, VS Code, JetBrains, Desktop, Web
- Context-aware codebase understanding
- Autonomous, collaborative, and **multi-agent** modes
- Multi-language support with **LSP code intelligence**
- Integrated git operations
- **Extensible**: Skills, Plugins, MCP, Hooks

Five Surfaces

- **CLI (Terminal)** — Full-featured, primary interface
- **VS Code Extension** — Inline diffs, @-mentions, plan review
- **JetBrains Plugin** — IntelliJ, PyCharm, WebStorm
- **Desktop App** — Native Mac/Windows with visual diffs, scheduling, connectors
- **Web (claude.ai/code)** — Browser-based, no local setup required

Local surfaces share: settings, CLAUDE.md, MCP servers, skills, and hooks. Web sessions only inherit repo-committed config — not user-level `~/.claude` settings.



Claude Desktop App

- **Three tabs:** Chat (general), Cowork (background agent), Code (interactive coding)
- **Visual diff review** with inline comments and "Review code" button
- **Live app preview** with embedded browser and auto-verify
- **Scheduled tasks** — persistent, local, survives restarts
- **Connectors** — GitHub, Slack, Linear, Notion, Google Calendar
- **Computer use** (research preview) — Claude controls screen on macOS/Windows
- **Parallel sessions** with automatic git worktree isolation

Claude Code on the Web

- **claude.ai/code** — Run tasks on Anthropic cloud infrastructure
- `--remote` **flag**: Start a web session from CLI: `claude --remote "Fix the auth bug"`
- **Diff view**: Review changes file-by-file before creating PRs
- **Auto-fix PRs**: Claude responds to CI failures and review comments automatically
 - **Caveat**: Replies post under your account — may trigger automation (Atlantis, Actions)
- `/teleport` : Pull web sessions back to your local terminal
- **Setup**: Connect GitHub, install Claude GitHub App, select environment
- Also accessible from **Claude iOS and Android apps**

Dispatch & Remote Control

Dispatch

- Send tasks from your **phone** to your **Desktop app**
- Dispatch routes dev tasks to Code tab, others to Cowork
- Push notification when done or needs approval

Remote Control

- Continue a **running CLI session** from phone or browser
- `claude --rc` to start with Remote Control enabled
- `/remote-control` to enable mid-session
- Session runs locally — web/mobile is just a window into it

Ultraplan

- **Cloud-based planning** for complex, codebase-wide changes
- Launch: `/ultraplan migrate the auth service from sessions to JWTs`
- Or include "ultraplan" in any prompt
- Claude drafts the plan in the cloud while **you keep working locally**
- **Browser review:** Inline comments, emoji reactions, outline navigation
- **Execute options:** Run in cloud (creates PR) or teleport back to terminal

Terminal status indicators:

- ◇ ultraplan – Claude is researching and drafting
- ◇ ultraplan needs your input – Clarifying question
- ◆ ultraplan ready – Plan ready to review in browser

Installation

- **Preferred:** Native binary installer:

```
curl -fsSL https://storage.googleapis.com/anthropic-releases/claude-cli/install.sh | bash
```

- Legacy (deprecated): `npm install -g @anthropic-ai/claude-code`
- Set API key: `export ANTHROPIC_API_KEY="your-key"`
- Verify: `claude --version`

Basic Usage

- Start Claude Code: `claude`
- Natural language prompts
- File-specific requests
- Multi-step workflows

Creating Projects from Scratch

- **Start from nothing:** Empty directory + idea = working application
- **Iterative development:** Concept → foundation → enhancements
- **Full-stack creation:** UI, logic, styling, tests in one session
- **Real example:** Our `lyrics-trainer` exercise started exactly this way

```
mkdir my-project && cd my-project && git init
claude
"Create a web app that displays song lyrics one line at a time
with Next, Previous, and Play buttons"
```

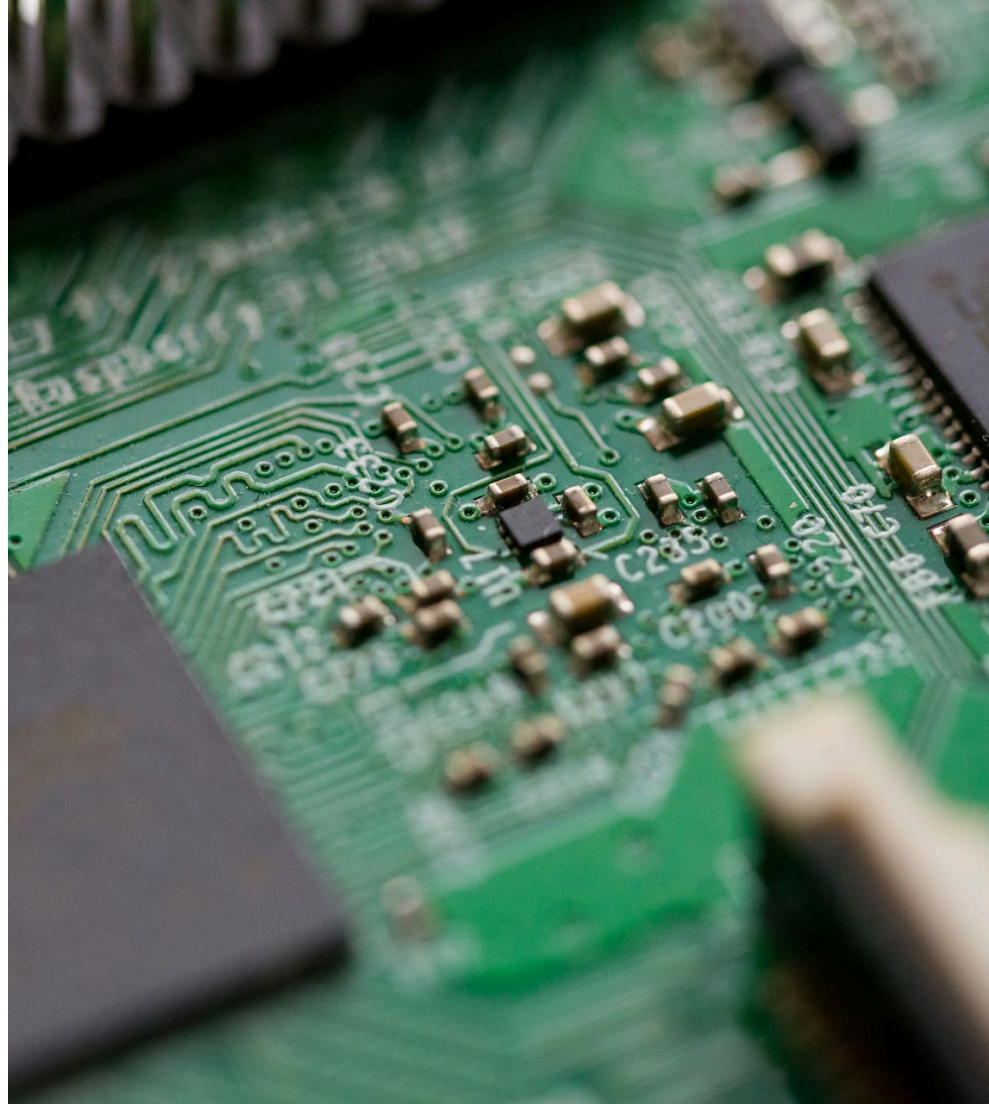
Operation Modes

- **Command Mode** (default) - Interactive conversation
- **Auto-Accept Mode** (Shift+Tab) - Autonomous execution
- **Plan Mode** (/plan or cycle with Shift+Tab) - Review plans before execution
- **Auto Mode** - Safety classifier eliminates permission prompts (opt-in)
- **Effort levels:** /effort low|medium|high to control reasoning depth
- **Model switch:** Alt+P / Option+P to change models mid-conversation

Core Productivity Features

**Get Productive
Immediately**

Essential features for daily
development work



Code Exploration

- Find files, functions, patterns
- Understand system architecture
- Trace dependencies
- Identify frameworks
- Reference specific files with `@path/to/file.java`

```
"Analyze the UserService class"  
"Explain @src/main/java/com/example/UserController.java"  
"How does @pom.xml configure Spring Boot?"
```

Test Generation

- Unit test creation
- Edge case identification
- Integration tests
- Mock object setup

```
"Create unit tests for the UserService"  
"Add tests for error scenarios"
```

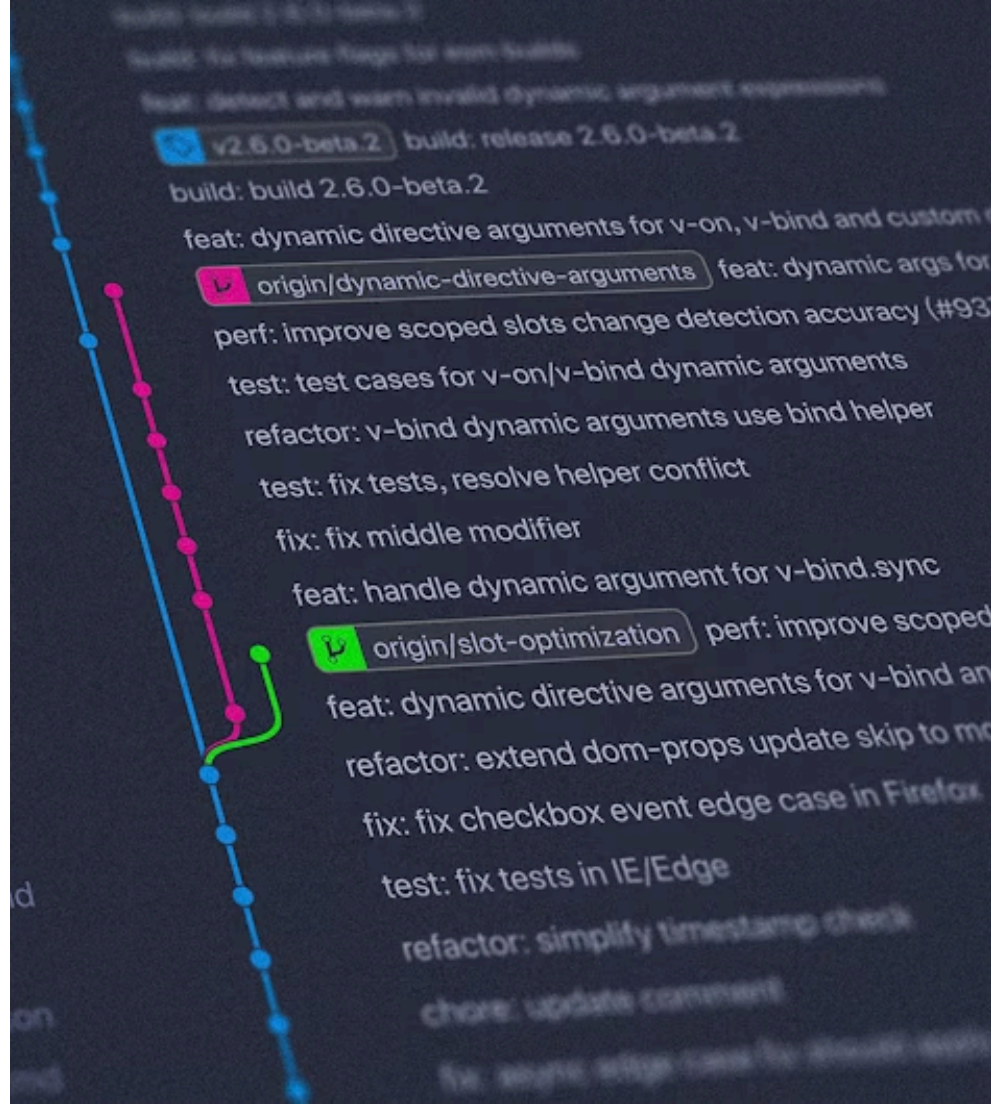
Refactoring Capabilities

- Multi-file operations
- Smart refactoring
- Pattern replacement
- Modern syntax adoption

Git Integration

- Commit message generation
- Branch management
- Merge conflict resolution
- Pull request creation

"Commit these changes with an appropriate message"
"Create a pull request for this feature"



Multi-Tool Workflows

- **Batch operations:** Call multiple tools in single response
- **Parallel execution:** Run git status + diff + log simultaneously
- **Performance optimization:** Reduces round-trips
- **Complex workflows:** Chain dependent operations

```
# Parallel git operations
"Show me git status, recent commits, and current diff"

# Multi-file analysis
"Check all test files and their coverage simultaneously"
```

💡 **Pro tip:** Request "in parallel" for faster execution

Documentation Generation

- Inline comments
- `README.md` files
- API documentation
- Architecture docs
- `CLAUDE.md` project configuration (covered later)

Debugging Workflows

- Error message analysis
- Stack trace navigation
- Configuration debugging
- Integration debugging



Essential Workflow Tools

Customize Your Experience

Session management and personalization

CLAUDE.md Files

- **Project memory:** `./CLAUDE.md` (shared with team)
- **User memory:** `~/.claude/CLAUDE.md` (personal preferences)
- **Rules directory:** `.claude/rules/` for organized project rules
- Auto-discovered up directory tree
- **Quick add:** Start input with `#` to add memory
- **Automatic memory:** Claude records and recalls across sessions
- **Commands:** `/memory` to edit, `/init` to bootstrap
- **Import files:** Use `@path/to/import` syntax

Custom Statusline

- **Configure context display:** `~/.claude/settings.json` or `/statusline`
- Shows git branch, status, working directory, custom info
- Keeps important context visible without asking
- Reduces repetitive status checks

```
{
  "statusline": {
    "items": [
      {"type": "git_branch"},
      {"type": "git_status"},
      {"type": "cwd"},
      {"type": "custom", "command": "node -v"}
    ]
  }
}
```

Perfect for teams wanting standardized context visibility

Custom Slash Commands (Skills-First)

- **Skills** (`.claude/skills/`) are the modern, recommended approach
- **Commands** (`.claude/commands/`) still work as a lightweight alternative
- Both: filename becomes command name, `$ARGUMENTS` for dynamic content
- **Project scope:** Shared with team via `.claude/skills/` or `.claude/commands/`
- **User scope:** Personal via `~/.claude/skills/` or `~/.claude/commands/`

Creating Custom Commands

- **Simple commands:** Just a markdown file — no frontmatter needed
- **Skills:** Add YAML frontmatter for model, effort, tools, paths
- **Use** `$ARGUMENTS` or `$0`, `$1` for dynamic content

```
# Quick command (lightweight)
mkdir -p .claude/commands
echo "Create service for $ARGUMENTS entity" > .claude/commands/service.md

# User command (personal, requires /user: prefix)
mkdir -p ~/.claude/commands
echo "Fix issue #$ARGUMENTS" > ~/.claude/commands/fix.md

# Usage: /service User or /user:fix 123
```

Real-world example:

```
# ~/.claude/commands/docs.md
Update both the README.md and CLAUDE.md files as appropriate.
If either file does not exist, please create it. Generate the
CLAUDE.md file as though the user invoked the init task.
```

Hooks & Automation

- **Event-driven workflow automation** with four hook types:
 - **Command hooks:** Run shell commands on events
 - **HTTP hooks:** POST JSON to a URL endpoint
 - **Prompt hooks:** Ask Claude for yes/no decisions
 - **Agent hooks:** Spawn subagents to verify conditions
- **Conditional hooks:** `if` field filters when hooks run (permission rule syntax)
- **PreToolUse:** Modify tool inputs, block dangerous operations
- **Security controls:** Validate and filter operations before they run
- **Configuration:** `~/.claude/settings.json` or `.claude/settings.json`

Hook Example: Security Validation

- **Pre-validate file edits** before they happen
- Block dangerous operations automatically
- Provide feedback that Claude can respond to

```
{  
  "hooks": {  
    "PreToolUse": [{  
      "type": "command",  
      "command": "validate-edit.sh",  
      "if": "Edit(**)"  
    }]  
  }  
}
```

Hook Example: Workflow Automation

- **Auto-format code** on file write
- Run linters, formatters, or validators
- Maintain code quality automatically

```
{  
  "hooks": {  
    "PreToolUse": [{  
      "type": "command",  
      "command": "prettier --write $FILE",  
      "if": "Write(**)"  
    }]  
  }  
}
```

Hook Example: Session Management

- **SessionEnd hooks** run when conversation ends
- Generate summary reports or logs
- Clean up resources or finalize work

```
{
  "hooks": {
    "SessionEnd": [{
      "type": "command",
      "command": "generate-session-report.sh"
    }]
  }
}
```

Important: Treat hook feedback as user input - Claude adjusts if blocked

Hook Events

- **Session:** SessionStart , SessionEnd , InstructionsLoaded
- **Tools:** PreToolUse , PostToolUse , PermissionRequest , PermissionDenied
- **Teams:** TeammateIdle , TaskCreated , TaskCompleted
- **Files:** FileChanged , CwdChanged , WorktreeCreate , WorktreeRemove
- **Config:** ConfigChange , Notification
- **Context:** PreCompact , PostCompact
- **MCP:** Elicitation , ElicitationResult
- **Agent-level hooks:** Skills and agents define their own hooks in frontmatter

Customizable Keybindings

- `/keybindings` command to configure keyboard shortcuts
- **Config file:** `~/.claude/keybindings.json`
- Remap any action to preferred key combinations
- `chat:newline` action for configurable multi-line input
- **Argument shorthand:** `$0` , `$1` in custom commands (not just `$ARGUMENTS`)

```
{  
  "chat:submit": "enter",  
  "chat:newline": "shift+enter",  
  "chat:switch_model": "alt+p",  
  "chat:open_external_editor": "ctrl+x ctrl+e"  
}
```

Key shortcuts: `Ctrl+B` (background), `Ctrl+X Ctrl+K` (kill agents), `Ctrl+X Ctrl+E` (external editor)

LSP: Code Intelligence

- **Language Server Protocol** integration for precise code navigation
- **Go to definition:** Jump to where symbols are defined
- **Find references:** Locate all usages across the codebase
- **Hover info:** Get type information and documentation
- **Call hierarchy:** Trace incoming and outgoing calls
- **Workspace symbols:** Search for symbols across the project
- Works with any configured LSP server (TypeScript, Java, Python, etc.)

Output Styles

- **Customize how Claude presents solutions** to match learning preferences
- **Built-in styles:** "Explanatory" and "Learning" modes
- **Custom styles:** Create your own in `~/.claude/output-styles/`
- **Configure via settings:** Set default style or switch per-session
- **Use cases:**
 - Educational contexts (verbose explanations)
 - Production work (concise, action-focused)
 - Code review (detailed analysis)
 - Quick fixes (minimal commentary)

Using Built-in Output Styles

- **Explanatory:** Verbose with detailed explanations
- **Learning:** Teaching-focused with step-by-step guidance
- **Configure in settings** or via command line

```
{  
  "outputStyle": "explanatory"  
}
```

```
claude --output-style learning
```

Creating Custom Output Styles

Create `~/.claude/output-styles/production.md` :

```
---  
name: Production  
description: Concise output for experienced developers  
---  
  
# Instructions for Claude  
  
- Be concise and action-focused  
- Skip explanations unless asked  
- Show code without lengthy preambles  
- Assume expert-level knowledge
```

Then use: `claude --output-style production`

Resuming Conversations

- `--continue` : Automatically resume most recent conversation
- `--resume` : Interactive picker showing conversation history with timestamps and message counts
- **Full history restored:** Complete message context maintained (even hundreds of messages)
- **Original settings preserved:** Model and configuration retained
- **Stored locally:** Complete conversation database maintained on your machine

```
# Continue most recent conversation
claude --continue

# Show conversation picker with details
claude --resume

# Continue with new prompt
claude --continue --print "Continue with my task"
```

Working with Images

- **Drag and drop** images into Claude Code window
- **Copy/paste** with `Ctrl+V` (not `Cmd+V` even on a Mac!)
- **Provide file path:** "Analyze this image: `/path/to/screenshot.png` "
- Analyze UI designs, error screenshots, diagrams
- Generate code from visual mockups
- Debug visual issues and layouts

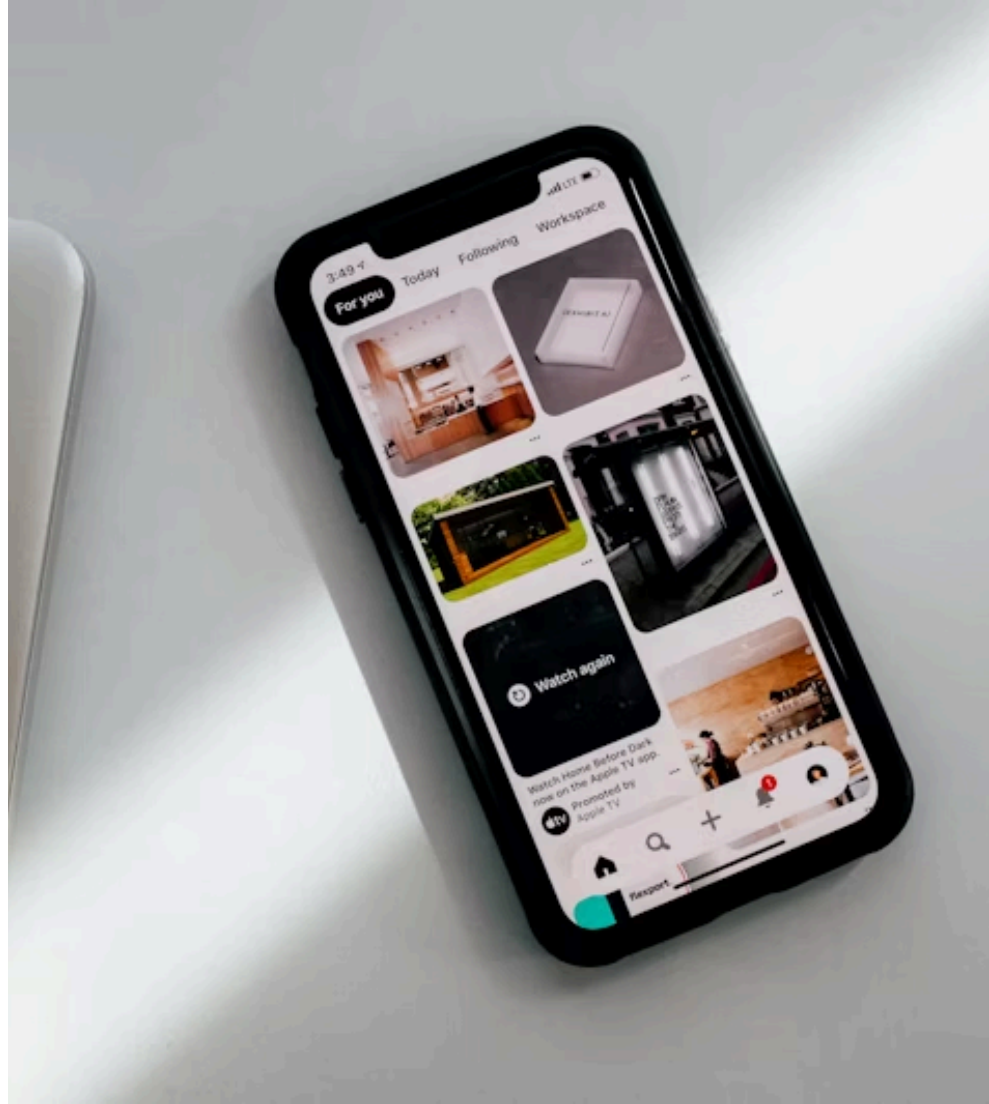
```
# Common image workflows
```

```
"Analyze this error screenshot and suggest fixes"
```

```
"Generate HTML/CSS for this UI mockup"
```

```
"Explain what this diagram shows"
```

```
"Convert this whiteboard sketch to code"
```



Jupyter & Data Science Support

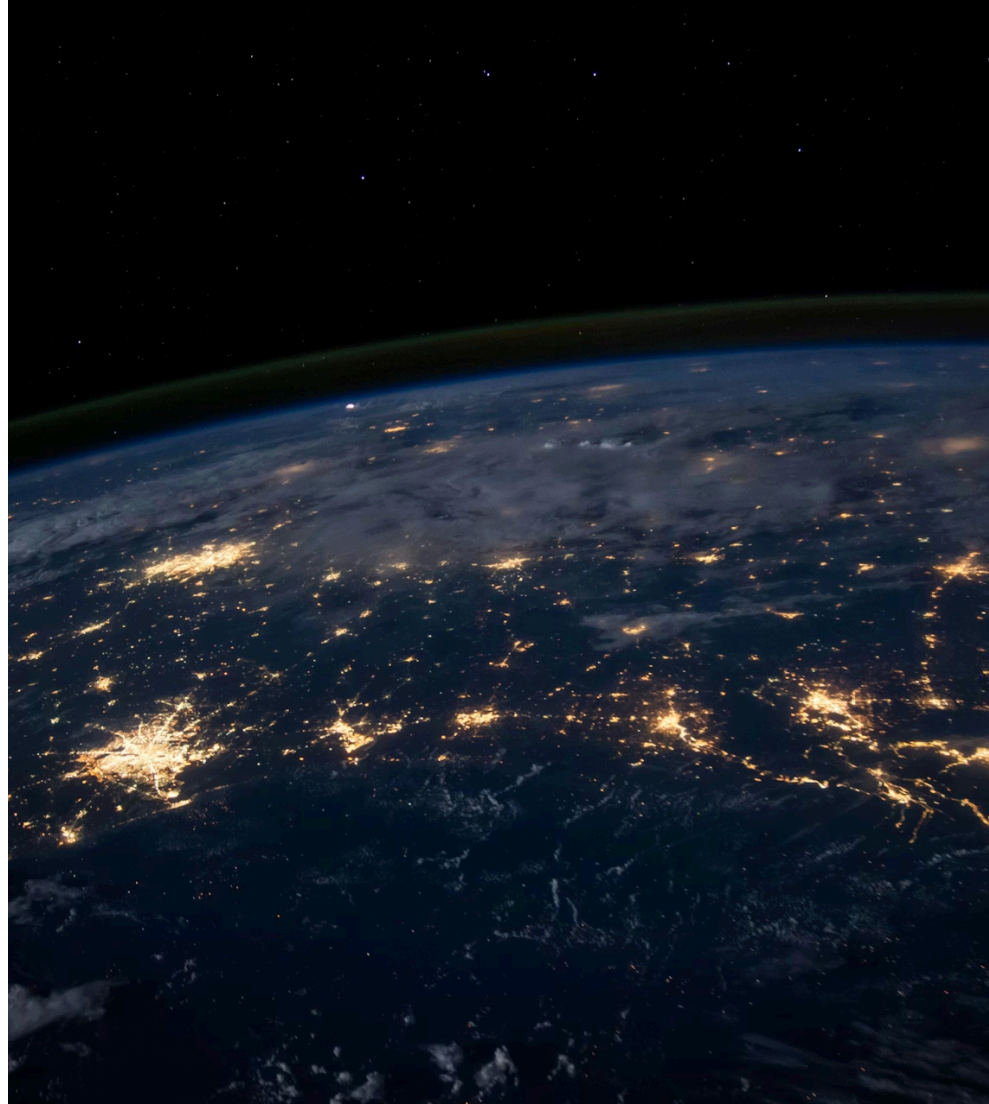
- **Read .ipynb files** with full cell outputs
- **Analyze notebooks:** Code, markdown, and visualizations
- **Edit notebook cells:** Use NotebookEdit tool
- **Data analysis workflows:** Process datasets and results
- **Visualization understanding:** Interpret charts and graphs

```
"Analyze this Jupyter notebook and explain the data pipeline"  
"Add error handling to the data processing cells"  
"Convert this notebook to a production Python script"
```

Advanced Features

Power User Capabilities

Complex features for sophisticated workflows



Skills: Persistent Domain Expertise

- **Modular capabilities** that extend Claude's functionality beyond the base model
- **Unified with slash commands** (v2.1): Skills and commands merged into one system
- **Three-tier loading system** for efficiency:
 - Metadata (always loaded): Name and description (~100 tokens)
 - Instructions (triggered): Main SKILL.md with procedures
 - Resources (on-demand): Scripts, templates, reference files
- **Hot-reload**: Edit SKILL.md and changes take effect immediately
- **Automatic activation** when contextually relevant
- **Progressive disclosure**: Load only what's needed for each task

Built-in Skills

Anthropic provides four production-ready Agent Skills:

-  **Excel (xlsx)**: Build spreadsheets, generate reports with charts
-  **Word (docx)**: Create and format professional documents
-  **PowerPoint (pptx)**: Create and edit presentations
-  **PDF (pdf)**: Generate formatted PDF documents and reports

Usage: These skills activate automatically when you reference relevant file types or request document creation

```
# Skills activate automatically
"Create a quarterly report spreadsheet with sales data"
"Generate a PDF proposal document with our company branding"
"Build a presentation deck for the product launch"
```

Creating Custom Skills

Skill Structure

```
~/ .claude/skills/my-skill/  
├─ SKILL.md           # Required: Instructions with YAML frontmatter  
├─ templates/         # Optional: Reusable templates  
├─ scripts/           # Optional: Helper scripts  
└─ reference/         # Optional: Documentation, schemas
```

Example SKILL.md

```
---  
name: Java Spring Generator  
description: Generate Spring Boot components following team patterns  
effort: high  
paths:  
  - "src/**/*.java"  
---
```

Instructions

When generating Spring Boot code:

1. Use constructor injection, not `@Autowired`
2. Follow package conventions: `controller/service/repository`

Plugins: Team-Wide Extensibility

- **Plugin system** (v2.0.12+) provides installable packages of commands, agents, hooks, and MCP servers
- **Plugin marketplace:** Discover and share team workflows
- **Repository-level config:** `extraKnownMarketplaces` for enterprise control
- **Management commands:**
 - `/plugin install <name>` - Install from marketplace
 - `/plugin enable/disable <name>` - Control active plugins
 - `/plugin marketplace` - Browse available plugins
 - `/plugin list` - View installed plugins

Plugin Use Cases

Enterprise Workflows

- Standardize code generation patterns across teams
- Enforce security review processes
- Automate compliance documentation
- Integrate with internal tools and APIs

Team Collaboration

- Share custom commands and agents
- Distribute MCP server configurations
- Maintain consistent development practices
- Onboard new team members faster

Example

```
# Install company's internal plugin
/plugin install acme-corp-standards

# Plugin provides:
# - Custom slash commands for service generation
```

Effort Levels

- `/effort low|medium|high` controls reasoning depth
- **Low:** Fast responses for simple tasks
- **Medium:** Balanced reasoning (default)
- **High:** Deep analysis for complex architecture
- **Keywords still work:** "think", "think harder", "ultrathink"
- Can set in skill frontmatter: `effort: high`

```
/effort high
```

```
# Or use keywords in your prompt
```

```
"Ultrathink about the best approach for implementing
```



Plan Mode

- Type `/plan` or cycle with `Shift+Tab` to activate
- Claude presents implementation plan before writing code
- Review strategy, approve, or modify approach
- Perfect for complex, multi-file changes
- **Uses the Plan subagent** behind the scenes
- `/ultraplan` : Cloud-based planning for codebase-wide changes (see Surfaces section)

Subagents: Specialized Task Handlers

- **Autonomous agents** that Claude launches for specialized tasks
- **Dynamic selection:** Claude chooses appropriate subagent automatically
- **Model selection:** Different subagents can use different models
- **Built-in types:**
 - **Plan:** Strategic task decomposition and planning
 - **Explore:** Fast codebase exploration and search
 - **General-purpose:** Full read/write access for complex work
- **Custom agents:** Define in `.claude/agents/` as markdown with YAML frontmatter
 - Specify: `model` , `tools` , `effort` , `hooks` , `permissionMode`
- **Background execution:** `Ctrl+B` to background, `Ctrl+X Ctrl+K` to kill
- **Worktree isolation:** `isolation: "worktree"` for safe parallel work

When Claude Uses Subagents

Automatic Activation

Claude launches subagents when tasks match specialized capabilities:

```
# Triggers Explore subagent (read-only, fast search)
"Find all API endpoints in this codebase"
"How does authentication work across the project?"

# Triggers Plan subagent
/plan or "Create a plan for adding OAuth"

# Triggers General-purpose subagent (full read/write)
"Generate comprehensive test coverage for UserService"
"Create API documentation for all REST endpoints"
```

You don't manage this - Claude handles subagent selection automatically for optimal results

Agent Teams (Research Preview)

- **Multi-agent orchestration:** A lead agent coordinates multiple teammates
- **Shared task list:** Tasks with dependencies, ownership, and status tracking
- **Independent context:** Each teammate gets its own context window
- **Inter-agent messaging:** Direct messages, broadcasts, and shutdown coordination
- **Automatic work distribution:** Teammates self-claim tasks as they finish

```
# Enable teams (research preview)
export CLAUDE_CODE_EXPERIMENTAL_AGENT_TEAMS=1
```

```
# Example prompt
"Create a team to refactor the auth module: one agent updates
the service layer, another updates tests, a third updates docs"
```

How Agent Teams Work

- **Lead agent** creates team, breaks work into tasks, assigns teammates
- **Teammates** work independently, report back, claim new tasks
- **Task coordination:** `blocks` / `blockedBy` dependencies prevent conflicts
- **Idle state is normal:** Teammates go idle between turns, wake on message
- **Hook events:** `TeammateIdle` , `TaskCompleted` for automation
- **Best for:** Large refactors, multi-file features, parallel code + test work

```
Lead Agent → creates tasks → assigns teammates
      ↑
      |
      | receives results ←
      |
      | (via shared task list + messages)
```

Background Agents & Worktree Isolation

- **Background agents:** Run tasks while you keep working
 - `Ctrl+B` to background a running agent
 - `Ctrl+X Ctrl+K` to kill all background agents
- **Worktree isolation:** Agents work in temporary git worktrees
 - `claude -w` or `claude --worktree` starts in isolated worktree
 - Subagents support `isolation: "worktree"` for safe parallel work
 - Changes are on a separate branch — merge when ready
- `/batch` **skill:** Parallel changes across many files using worktrees
- **Combined power:** Background + worktree = fearless parallel development

Session Management & Cross-Surface Workflows

- `--remote` : Start a cloud session: `claude --remote "Fix the auth bug"`
- `/teleport` : Pull a web session back to your terminal
- **Dispatch**: Send tasks from phone → Desktop app (Pro/Max)
- **Named sessions**: `/rename my-feature` for easy identification
- **PR-linked sessions**: `claude --from-pr 123` resumes PR context
- **Session branching**: `/branch` to branch a conversation
- **Resume picker**: `claude --resume` shows up to 50 recent sessions
- **Auto-memory**: Claude records and recalls context across sessions

Auto-Memory

- Claude **automatically records and recalls** memories as it works
- Stored at `~/.claude/projects/<project>/memory/`
- `MEMORY.md` index file loaded at session start (first 200 lines)
- Topic files loaded on demand when relevant
- `/memory` command to view and manage memories
- Memory types: user preferences, feedback, project context, references
- Subagents can maintain their own auto memory
- Toggle with `autoMemoryEnabled` setting

Auto Mode

- **Eliminates permission prompts** via a background safety classifier
- Classifier reviews each action and allows/blocks automatically
- **Different from Auto-Accept** (`Shift+Tab`): Auto Mode is intelligent, not blanket
- **Allows**: Local file ops, dependency installs, read-only HTTP, pushing to current branch
- **Blocks**: Downloading + executing code, production deploys, force pushes, IAM changes
- **Requirements**: Team/Enterprise/API plan, Sonnet 4.6 or Opus 4.6
- Enable: `--enable-auto-mode` or cycle with `Shift+Tab`

Scheduled Tasks: Three Tiers

	/loop	Desktop	Cloud
Runs on	Your machine	Your machine	Anthropic cloud
Requires open session	Yes	No	No
Persistent	No	Yes	Yes
Local file access	Yes	Yes	No
Min interval	1 minute	1 minute	1 hour

Using Scheduled Tasks

- `/loop` — Session-scoped polling: `/loop 5m check if the deployment finished`
- **Desktop** — Persistent local tasks that survive restarts
- **Cloud** — Always-on via `/schedule` — runs even with your machine off
- Cloud tasks clone the repo fresh each run (default branch)

Channels (Research Preview)

- **Push events** from external sources into running Claude Code sessions
- Supported: **Telegram, Discord, iMessage** (via plugins)
- Two-way: Claude reads events and replies back
- Use cases: chat bridge from phone, CI/monitoring webhooks
- `--channels` flag to enable: `claude --channels plugin:telegram@claude-plugins-official`
- Requires claude.ai login
- Enterprise: admin must enable `channelsEnabled`

Model Context Protocol (MCP)

- Standard protocol for AI-to-system connections
- Tool integration (APIs, databases, services)
- Context enhancement for better AI responses
- Security controls and permissions
- **MCP Tool Search** (default since v2.1): Lazy-loads tools on demand
 - Reduces context usage by ~95% with many MCP tools
 - Tools discovered automatically when needed

MCP Configuration

```
# Import from Claude Desktop
claude mcp add-from-claude-desktop

# Add remote server (HTTP)
claude mcp add --transport http context7 https://mcp.context7.com/mcp

# Add local server
claude mcp add context7 -- npx -y @upstash/context7-mcp
```

- **Project config:** `.mcp.json` in project root
- **Management:** `claude mcp list`, `/mcp enable|disable`

MCP Server Examples

- **GitHub MCP** - Repository operations, issues, PRs
- **Context7** - Downloads latest API docs and examples for modern code
- **Docker MCP Toolkit** - Container management and operations
- **Playwright MCP** - UI test generation and browser automation
- **Heroku MCP** - Deployment and app management

Setting Up MCP Servers

- Interactive setup: `claude mcp`
- Local servers: Full configuration control
- Remote servers: OAuth authentication, zero maintenance
- Docker MCP Toolkit: `docker mcp gateway run`

```
# List existing MCP servers
claude mcp list

# Add local server
claude mcp add my-server -e API_KEY=123 -- /path/to/server

# Add remote server (HTTP)
claude mcp add --transport http remote-server https://example.com/mcp

# Add Docker MCP toolkit
claude mcp add docker-mcp docker mcp gateway run
```

Claude Code SDKs

- **Available SDKs:** TypeScript, Python, Command Line
- **Build AI-powered coding assistants** into your workflows
- **Multi-turn conversations** and session management
- **Custom system prompts** and flexible I/O formats
- **MCP integration** for extended capabilities

```
# Command line usage
claude -p "Write a function to calculate Fibonacci numbers"
claude -p "Generate a hello world function" --output-format json
```

```
// TypeScript SDK
import { query } from "@anthropic-ai/claude-code";

for await (const message of query({
  prompt: "Write a haiku about foo.py",
  options: { maxTurns: 3 }
})) {
  // Process messages
}
```


SDK: CI/CD Integration

- **Automate code reviews** in pull requests
- **JSON output** for parsing results
- **Integrate with GitHub Actions**, GitLab CI, etc.

```
# GitHub Actions example
- name: AI Code Review
  run: |
    claude -p "Review PR changes" --output-format json > review.json
```

SDK: Git Hooks

- **Pre-commit security checks** before code is committed
- **Limit tools** for focused, fast execution
- **Fail-fast** on security issues

```
#!/bin/bash
# .git/hooks/pre-commit
claude -p "Check for security issues" --allowed-tools read,grep
```

VS Code Extension

- **Native IDE integration** bringing Claude Code into your editor
- **In-editor experience:** Work with Claude without leaving VS Code
- **Context-aware:** Accesses your workspace files and settings
- **All Claude Code features:** Skills, MCP, custom commands available
- **Installation:** Search "Claude Code" in VS Code Extensions marketplace

VS Code Extension Features

Integrated Workflow

- Reference files with `@` syntax directly in VS Code
- View diffs and approve changes inline
- **Session management** with AI-generated titles, rename, fork
- **Plan view panel** with full markdown and commenting
- **MCP server management** via `/mcp` in extension
- **Compaction display** as collapsible card

Also Available

- **JetBrains Plugin:** IntelliJ, PyCharm, WebStorm — same core features



Management & Control

Monitor and Control

Cost, context, and permission management

Cost Monitoring

- Use `/cost` command to check usage
- Shows current usage and limits
- Pro Plan: Displays prompt count vs limit
- Max Plans: Shows monthly usage summary
- Limits reset every 5 hours
- Plan ahead for intensive work sessions

```
# Check your current usage
```

```
/cost
```

```
# Example output (Pro Plan):
```

```
#  Cost information:
```

```
#   - Input tokens: 1,245
```

```
#   - Output tokens: 3,782
```

```
#   - Total cost: $0.076
```

```
# Example output (Max Plan):
```

```
# With your Claude Max subscription, no need to monitor cost
```

```
# - your subscription includes Claude Code usage
```

Context Management

- Use `/compact` command to compress conversation history
- Claude Code automatically compacts when context limit approaches
- Warning message appears before auto-compaction
- Preserves essential information while reducing token usage
- Manual compaction gives you control over timing

```
# Manually compact the conversation  
/compact
```

```
# Warning message example:  
# ⚠ Context limit approaching. Auto-compacting in next response  
# to preserve conversation history and continue working.
```

Smart Context Management

- **Auto-compaction:** Automatic when approaching limits
- **Warning messages:** Advance notice before compaction
- **Preservation strategy:** Keeps essential information
- **Manual control:** Use `/compact` proactively
- **Long conversations:** Maintains context over hundreds of messages

```
# Warning you'll see:
```

```
⚠ Context limit approaching. Auto-compacting in next response  
to preserve conversation history and continue working.
```

```
# Proactive management:
```

```
/compact # Compress conversation manually
```

Configuring Permissions

- **Fine-grained control** over Claude Code's capabilities
- Use `/permissions` **UI** to manage tool permissions
- **Allow/Deny rules** for specific tools and actions
- **Enterprise policies** for organization-wide control
- **Permission precedence:** Enterprise → CLI → Project → User

```
# Example permission rules
Bash(npm run test:*)      # Allow npm test commands
Edit(docs/**)             # Allow editing docs directory
Read(src/*)               # Allow reading source files

# Access permissions UI
/permissions
```

Permission Modes

Six modes (cycle with `Shift+Tab`)

- **Default:** Prompts for each action
- **Accept Edits:** Auto-approves file edits
- **Plan:** Read-only exploration, no edits
- **Auto:** Safety classifier decides (Team/Enterprise/API)
- **Don't Ask:** Only pre-approved tools run (CI/CD)
- **Bypass Permissions:** No prompts at all

```
# Quick permission profiles via aliases
alias claude-dev='claude --allowed-tools all'
alias claude-review='claude --allowed-tools read,grep'
alias claude-safe='claude --disabled-tools bash,webfetch'
alias claude-ci='claude --allowed-tools bash,git,test'
```



Team & Best Practices

Collaborate Effectively

Team workflows and professional practices

Git Worktrees for Parallel Sessions

- **Built-in worktree support:** `claude -w` or `claude --worktree`
 - Automatically creates isolated worktree + new branch
 - Prompts to keep or remove on session exit
- **Manual worktrees** also work for full control
- Share git history while isolating working files
- Perfect for multi-feature development

```
# Built-in (preferred) - automatic isolation
claude -w                # Start in new worktree
claude --worktree        # Same thing

# Manual worktrees for full control
git worktree add ../project-feature-a -b feature-a
cd ../project-feature-a && claude

# Manage worktrees
git worktree list
git worktree remove ../project-feature-a
```

Effective Prompting & Best Practices

Effective Prompting

- **Be specific** about what you want to achieve
- **Provide context** about your goals and constraints
- **Use iterative refinement** for complex tasks
- **Include examples** when possible to show desired patterns

Best Practices

- **Create a git branch first** for any significant changes
- **Commit checkpoints regularly** during development
- **Review all AI-generated code** before accepting
- **Test generated code** thoroughly

Troubleshooting & Configuration

System Health Check

```
claude /doctor # Diagnose installation issues
```

Global Configuration

```
claude config set -g model claude-sonnet-4-6  
claude config set -g verbose true  
claude config set -g max_conversation_turns 10
```

Check Current Settings

```
claude config list # View all settings  
echo $ANTHROPIC_API_KEY # Verify API key
```

Security & Permissions

Security-Focused Aliases

```
# Safe mode - limited tools
alias claude-safe='claude --allowed-tools read,write,edit'

# Review mode - read-only
alias claude-review='claude --allowed-tools read,grep'

# Development mode - all tools
alias claude-dev='claude --allowed-tools all'
```

Tool Restrictions

```
# Disable specific tools
claude --disabled-tools bash,webfetch

# Allow only specific tools
claude --allowed-tools read,write,edit,task
```

Common Issues: Installation

- **Command not found** → Check PATH: `npm list -g @anthropic/claude-code`
- **Permission denied** → Use correct npm prefix or sudo
- **Windows users** → WSL is required (not native Windows)

```
# Preferred: Native binary installer
curl -fsSL https://storage.googleapis.com/anthropic-releases/claude-cli/install.sh | bash

# Legacy npm (deprecated but still works)
npm uninstall -g @anthropic/claude-code
npm install -g @anthropic/claude-code
```

Common Issues: Runtime

- **API key not found** → Set `ANTHROPIC_API_KEY` environment variable
- **Rate limits** → Use `/cost` to monitor usage
- **Context too large** → Use `/compact` to reduce conversation size

```
# Alternative installation: Direct binary
```

```
curl -fsSL https://storage.googleapis.com/anthropic-releases/claude-cli/install.sh | bash
```

Development Process

- Start with clean git state
- Generate tests if none exist
- Commit checkpoints regularly
- Use Claude for git workflows (commits, issues, merges)
- Use git worktrees for parallel sessions on different branches
- Review changes before accepting
- Test generated code thoroughly

Team Collaboration

- Share `CLAUDE.md` configurations
- Document successful patterns
- Review AI-generated code together
- Establish team conventions

New Commands: Workflow

Command	Description
<code>/effort low medium high</code>	Set reasoning depth
<code>/plan</code>	Enter Plan Mode from prompt
<code>/ultraplan</code>	Cloud-based planning session
<code>/batch</code>	Parallel changes across codebase
<code>/loop 5m prompt</code>	Recurring prompt execution
<code>/memory</code>	View and manage auto-memory

New Commands: Utilities

Command	Description
<code>/branch</code>	Branch the conversation (was <code>/fork</code>)
<code>/copy N</code>	Copy Nth-latest response to clipboard
<code>/context</code>	Get context optimization suggestions
<code>/color</code>	Set prompt-bar color for session
<code>/powerup</code>	Interactive feature lessons

Removed: `/tag` , `/vim` (use `/config`). **Deprecated:** `/output-style` (use `/config`)

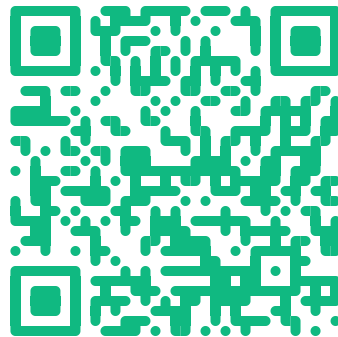
Quick Access

Claude Code Docs



docs.anthropic.com/claude-code

Course Repository



github.com/kousen/claude-code-training

Important Links



Claude Code Documentation

<https://docs.anthropic.com/en/docs/claude-code>



Official GitHub Repository

<https://github.com/anthropics/claude-code>



Course Source Code & Exercises

<https://github.com/kousen/claude-code-training>



Support & Issues

<https://github.com/anthropics/claude-code/issues>

Thank You!

Questions?



Kenneth Kousen

Author, Speaker, Java & AI Expert

kousenit.com | @kenkousen